

Лабораторная работа № 6. Решение моделей в непрерывном и дискретном времени

Абакумова О. М.

Российский университет дружбы народов, Москва, Россия

Информация

- Абакумова Олеся Максимовна
- Студентка
- Российский университет дружбы народов
- 1132220832@pfur.ru
- <https://github.com/omabakumova>



Цель работы

Основной целью работы является освоение специализированных пакетов для решения задач в непрерывном и дискретном времени.

Задание

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

Выполнение лабораторной работы

Решение обыкновенных дифференциальных уравнений

Напомним, что обыкновенное дифференциальное уравнение (ОДУ) описывает изменение некоторой переменной u :

$$u'(t) = f(u(t), p, t)$$

,

где

$$f(u(t), p, t)$$

— нелинейная модель (функция) изменения $u(t)$ с заданным начальным значением

$$u(t_0) = u_0$$

, p — параметры модели, t — время. Для решения обыкновенных дифференциальных уравнений (ОДУ) в Julia можно использовать пакет

Решение обыкновенных дифференциальных уравнений

Рассмотрим пример использования этого пакета для решение уравнения модели экспоненциального роста, описываемую уравнением

$$u'(t) = au(t), \quad u(0) = u_0$$

,

где a — коэффициент роста. Предположим, что заданы следующие начальные данные $a = 0,98, u(0) = 1,0, t \in [0; 1,0]$.

Аналитическое решение модели имеет вид:

$$u(t) = u_0 \exp(at)$$

Решение обыкновенных дифференциальных уравнений

```
In [1]: # подключаем необходимые пакеты:
import Pkg
Pkg.add("DifferentialEquations")
```

Installed	SciMLBase	v2.127.0
Installed	SimpleUnPack	v1.1.0
Installed	OrdinaryDiffEqFIRK	v1.16.0
Installed	NonlinearSolveQuasiNewton	v1.11.0
Installed	BoundaryValueDiffEqMIRKN	v1.8.0
Installed	NLSolve	v4.5.1
Installed	ADTypes	v1.19.0
Installed	OrdinaryDiffEqNonlinearSolve	v1.15.0
Installed	BracketingNonlinearSolve	v1.6.0
Installed	LinearSolve	v3.47.0
Installed	SimpleNonlinearSolve	v2.9.0
Installed	IFElse	v0.1.1
Installed	NonlinearSolveSpectralMethods	v1.6.0
Installed	LineSearch	v0.1.4
Installed	ThreadingUtilities	v0.5.5
Installed	DiffEqNoiseProcess	v5.24.1
Installed	JumpProcesses	v9.19.1
Installed	ManualMemory	v0.1.8
Installed	SparseConnectivityTracer	v1.1.3
Installed	SparseMatrixColonne	v0.4.12

```
In [7]: using DifferentialEquations
```

Рис. 1: Установка и подключение пакета `diffrentialEquations.jl`

Решение обыкновенных дифференциальных уравнений

```
In [4]: # задаём описание модели с начальными условиями:  
a = 0.98  
f(u,p,t) = a*u  
u0 = 1.0  
# задаём интервал времени:  
tspan = (0.0,1.0)
```

```
Out[4]: (0.0, 1.0)
```

```
In [5]: # решение:  
prob = ODEProblem(f,u0,tspan)  
sol = solve(prob)
```

```
Out[5]: retcode: Success  
Interpolation: 3rd order Hermite  
t: 5-element Vector{Float64}:  
 0.0  
 0.10042494449239292  
 0.3521860297865888  
 0.6934436122197829  
 1.0  
u: 5-element Vector{Float64}:  
 1.0  
 1.1034222047865465  
 1.412190871348492  
 1.9730384457359196  
 2.664456142481427
```

Рис. 2: Численное решение

Решение обыкновенных дифференциальных уравнений

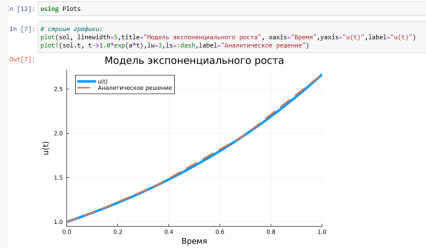


Рис. 3: Модель экспоненциального роста

Решение обыкновенных дифференциальных уравнений

При построении одного из графиков использовался вызов `sol.t`, чтобы захватить массив моментов времени. Массив решений можно получить, воспользовавшись `sol.u`.

Если требуется задать точность решения, то можно воспользоваться параметрами `abstol` (задаёт близость к нулю) и `reltol` (задаёт относительную точность). По умолчанию эти параметры имеют значение `abstol = 1e-6` `reltol = 1e-3`.

Решение обыкновенных дифференциальных уравнений

```

In [8]: # создаю нулевой пестик
sol = solve(p0, pstat=1, n=1, ncol=10)
print(sol)

# создаю заголовок
plot(sol, l=2, color="black", title="График экспоненциального роста", xaxis="Time", yaxis="U(t)", label="Численное решение")
plot(sol, t=1, l=1, xgrid=True, l=3, d=3, color="red", label="Аналитическое решение")

```

Рис. 4: Задание точности решения

Решение обыкновенных дифференциальных уравнений

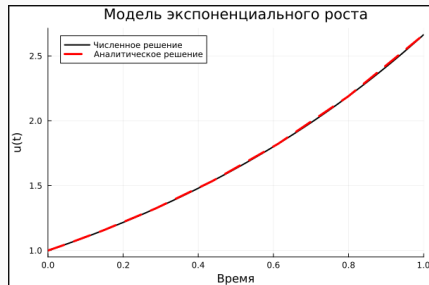


Рис. 5: Модель экспоненциального роста

Решение обыкновенных дифференциальных уравнений

Динамической системой Лоренца является нелинейная автономная система обыкновенных дифференциальных уравнений третьего порядка:

$$\begin{cases} \dot{x} = \sigma(y - x), \\ \dot{y} = \rho x - y - xz, \\ \dot{z} = xy - \beta z, \end{cases}$$

где σ , ρ и β — параметры системы (некоторые положительные числа, обычно указывают $\sigma = 10$, $\rho = 28$ и $\beta = 8/3$).

Решение обыкновенных дифференциальных уравнений

```
In [9]: # задаём описание модели:
function lorenz!(du,u,p,t)
    σ,ρ,β = p
    du[1] = σ*(u[2]-u[1])
    du[2] = u[1]*(ρ-u[3]) - u[2]
    du[3] = u[1]*u[2] - β*u[3]
end

Out[9]: lorenz! (generic function with 1 method)

In [10]: # задаём начальное условие:
u0 = [1.0,0.0,0.0]
# задаём значения параметров:
p = (10,28,8/3)
# задаём интервал времени:
tspan = (0.0,100.0)
# решение:
prob = ODEProblem(lorenz!,u0,tspan,p)
sol = solve(prob)

Out[10]: retcode: Success
Interpolation: 3rd order Hermite
t: 1303-element Vector{Float64}:
 0.0
 3.5678604836301404e-5
 0.0003924646531993154
 0.0032623945564751056
 0.009058054066400603
 0.016956441391185063
 0.02768991931434415
 0.04185629724071979
 0.060240340738233165
```

Рис. 6: Численное решение

Решение обыкновенных дифференциальных уравнений

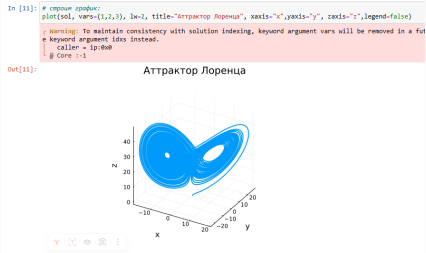


Рис. 7: Аттрактор Лоренца

Решение обыкновенных дифференциальных уравнений

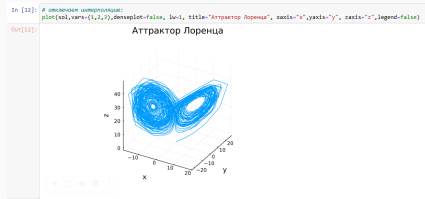


Рис. 8: Аттрактор Лоренца (интерполяция отключена)

Модель Лотки–Вольтерры

Модель Лотки–Вольтерры описывает взаимодействие двух видов типа «хищник – жертва»:

$$\begin{cases} \dot{x} = (\alpha - \beta y)x, \\ \dot{y} = (-\gamma + \delta x)y, \end{cases}$$

где x — количество жертв, y — количество хищников, t — время, $\alpha, \beta, \gamma, \delta$ — коэффициенты, отражающие взаимодействия между видами (в данном случае α — коэффициент рождаемости жертв, γ — коэффициент убыли хищников, β — коэффициент убыли жертв в результате взаимодействия с хищниками, δ — коэффициент роста численности хищников).

Модель Лотки-Вольтерры

```
In [13]: # подключаем необходимые пакеты:
import Pkg
Pkg.add("ParameterizedFunctions")
```

Resolving package versions...
Installed Unitful ————— v1.25.1
Installed MultivariatePolynomials — v0.5.13
Installed Tricks ————— v0.1.13
Installed MutableArithmetics ——— v1.6.7
Installed Bijections ————— v0.2.2
Installed FindFirstFunctions ——— v1.4.2
Installed DomainSets ————— v0.7.16
Installed JuliaFormatter ————— v2.2.0
Installed CompositeTypes ————— v0.1.4
Installed JuliaSyntax ————— v0.4.10
Installed Glob ————— v1.3.1
Installed DispatchDoctor ————— v0.4.26
Installed DynamicPolynomials ——— v0.6.4
Installed Symbolics ————— v6.57.0
Installed MLStyle ————— v0.4.17
Installed CommonMark ————— v0.9.1
Installed ModelingToolkit ——— v10.29.0
Installed ParameterizedFunctions — v5.19.0
Installed AbstractTrees ————— v0.4.5

```
In [6]: using ParameterizedFunctions
```

Рис. 9: Подключение пакета ParameterizedFunctions

Модель Лотки–Вольтерры

[illegible]

Рис. 10: Численное решение

Модель Лотки-Вольтерры

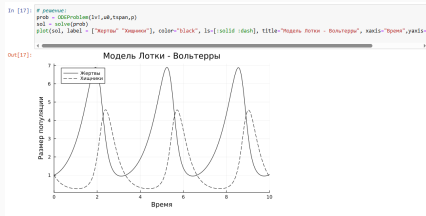


Рис. 11: Модель Лотки-Вольтерры: динамика изменения численности популяций

Модель Лотки-Вольтерры

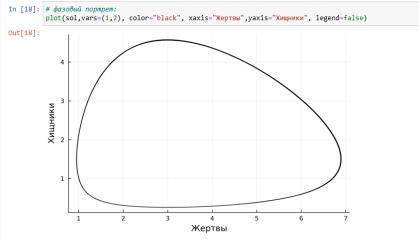


Рис. 12: Модель Лотки-Вольтерры: фазовый портрет

Задания для самостоятельного выполнения

1. Реализовать и проанализировать модель роста численности изолированной популяции (модель Мальтуса):□

$$\dot{x} = ax, \quad a = b - c,$$

где $x(t)$ — численность изолированной популяции в момент времени t , a — коэффициент роста популяции, b — коэффициент рождаемости, c — коэффициент смертности. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией).

Задания для самостоятельного выполнения

```
In [20]: # 1. Модель Мальтуса (экспоненциальный рост)
function malthus_model(du, u, p, t)
    k = u[t]
    a = p[1]
    du[t] = a * k
end

Out[20]: malthus_model! (generic function with 1 method)

In [21]: # Параметры для модели Мальтуса
a = 0.1 # коэффициент роста (рождаемость 0.15, смертность 0.05)
u0_malthus = [10.0] # начальная численность популяции
tspan_malthus = (0.0, 50.0)
p_malthus = [a]

# Решение модели Мальтуса
prob_malthus = ODEProblem(malthus_model, u0_malthus, tspan_malthus, p_malthus)
sol_malthus = solve(prob_malthus, Tsitsis())

Out[21]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 12-element Vector{Float64}:
 0.0
 0.15849248898571244
 1.562544352948681
 4.229216240948622
 7.818118689793625
12.091558089145196
17.12383945146074
22.493927633394653
29.2460680589828527
36.42928768224435
44.8628895487791
50.0
u: 12-element Vector{Vector{Float64}}:
 [10.0]
 [10.159755144258859]
 [11.091238316816869]
 [15.417553734758194]
 [21.83888857762813]
 [30.586942451892739]
 [42.198698565943]
 [58.08918312511201]
 [80.23989577497495]
 [109.8306480726474]
 [151.1956659558941]
 [206.8928612236227]

In [22]: # График для модели Мальтуса
p1 = plot(sol_malthus, label="численность популяции",
          xlabel="Время", ylabel="Численность",
          title="Модель Мальтуса: экспоненциальный рост",
          linewidth=3, legend=:topleft)
annotate!(p1, 30, 100, text("n = 50", 10))
```

Рис. 13: Модель Мальтуса

Задания для самостоятельного выполнения

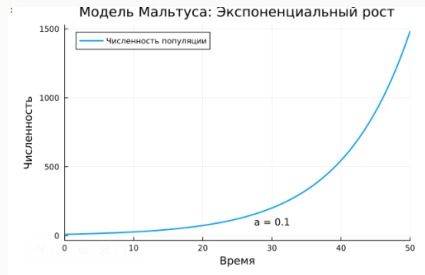


Рис. 14: График модели Мальтуса

Задания для самостоятельного выполнения

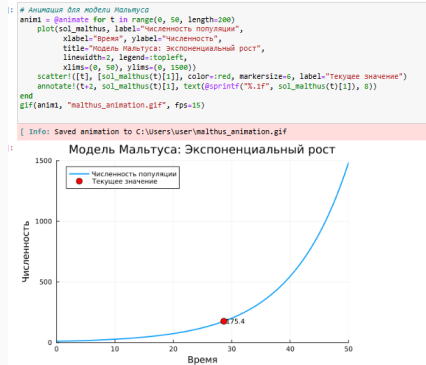


Рис. 15: Анимация модели Мальтуса

Задания для самостоятельного выполнения

2. Реализовать и проанализировать логистическую модель роста популяции, заданную уравнением:

$$\dot{x} = rx \left(1 - \frac{x}{k}\right), \quad r > 0, \quad k > 0,$$

r — коэффициент роста популяции, k — потенциальная ёмкость экологической системы (предельное значение численности популяции). Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией).

Задания для самостоятельного выполнения

```
In [24]: # 2. Логистическая модель
function logistic_model(du, u, p, t)
    k = u[1]
    r, k = p
    du[1] = r * u * (1 - u/k)
end

Out[24]: logistic_model (generic function with 1 method)

In [25]: # Параметры для логистической модели
r = 0.2 # коэффициент роста
k = 500.0 # емкость среды
u0_logistic = [10.0] # начальная численность популяции
tspan_logistic = (0.0, 100.0)
p_logistic = [r, k]

# Решение логистической модели
prob_logistic = ODEProblem(logistic_model, u0_logistic, tspan_logistic, p_logistic)
sol_logistic = solve(prob_logistic, Tsitsis())

Out[25]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 20-element Vector{Float64}:
 0.0
 0.1385343499475166
 0.9861418836884212
 2.4862334824599643
 4.317341221889021
 6.682398587816193
 9.294648327798243
12.447195676651724
16.05880801227316
20.457695921165335
26.06310153773248
31.085844027481574
37.00038853544241
42.5828080863126
49.38271656444827
56.74524741129299
66.04020150161459
77.26303384728847
91.9540259788935
100.0
u: 20-element Vector{Vector{Float64}}:
 [10.0]
 [10.275169288997861]
 [12.1273411185667547]
 [16.232742072928964]
 [23.08066633443723]
 [35.582648601227705]
 [57.896334343451276]
 [98.71875984961598]
 [169.00050022664556]
 [274.8832334489878]
```

Рис. 16: Логистическая модель роста популяции

Задания для самостоятельного выполнения

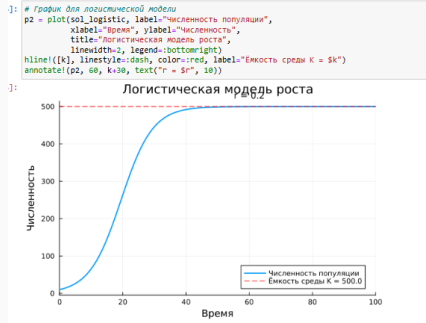


Рис. 17: График логистической модели роста популяции

Задания для самостоятельного выполнения

```
1: # Анимация для логистической модели
anim2 = @animate for t in range(0, 100, length=200)
    plot(sol_logistic, label="Численность популяции",
        xlabel="Время", ylabel="Численность",
        title="Логистическая модель роста",
        linewidth=2, legend=:bottomright,
        xlims=(0, 100), ylims=(0, 600))
    hline!([K], linestyle=:dash, color=:red, label="Ёмкость среды K = 5k")
    scatter!([t], [sol_logistic(t)[1]], color=:red, markersize=6, label="Текущее значение")
    annotate!(t+s, sol_logistic(t)[1], text{@sprintf("%.1f", sol_logistic(t)[1]), s})
end
gif(anim2, "logistic_animation.gif", fps=15)
```

[Info: Saved animation to C:\Users\user\logistic_animation.gif

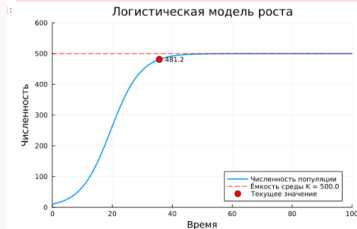


Рис. 18: Анимация графика логистической модели роста популяции

3. Реализовать и проанализировать модель эпидемии Кермака–Маккендрика (SIR- модель):

$$\dot{s} = -\beta i s,$$

$$\dot{i} = \beta i s - \nu i,$$

$$\dot{r} = \nu i$$

Задания для самостоятельного выполнения

где $s(t)$ — численность восприимчивых к болезни индивидов в момент времени t , $i(t)$ — численность инфицированных индивидов в момент времени t , $r(t)$ — численность переболевших индивидов в момент времени t , β — коэффициент интенсивности контактов индивидов с последующим инфицированием, ν — коэффициент интенсивности выздоровления инфицированных индивидов. Численность популяции считается постоянной, т.е. $\dot{s} + \dot{i} + \dot{r} = 0$. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией).

Задания для самостоятельного выполнения

```
]]: # 3. Модель эпидемии SIR
function sir_model!(du, u, p, t)
    s, i, r = u
    β, v = p

    du[1] = -β * i * s # ds/dt
    du[2] = β * i * s - v * i # di/dt
    du[3] = v * i # dr/dt
end

]: sir_model! (generic function with 1 method)

]: # Параметры для SIR модели
β = 0.0003 # коэффициент заражения
v = 0.1 # коэффициент выздоровления
u0_sir = [990.0, 10.0, 0.0] # [восприимчивые, инфицированные, переболевшие]
tspan_sir = (0.0, 200.0)
p_sir = [β, v]

# Решение SIR модели
prob_sir = ODEProblem(sir_model!, u0_sir, tspan_sir, p_sir)
sol_sir = solve(prob_sir, Tsitsis())

]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 33-element Vector{Float64}:
 0.0
 0.0014141421188071605
 0.015555563306878765
 0.1569697751875948
 0.7273501964876006
 1.810816243524848
 3.3104998683571587
 5.233807572206311
 7.632226944539838
10.541185813363883
14.051001968636427
18.235777688738175
23.355755602185027
```

Рис. 19: Модель эпидемии SIR

Задания для самостоятельного выполнения

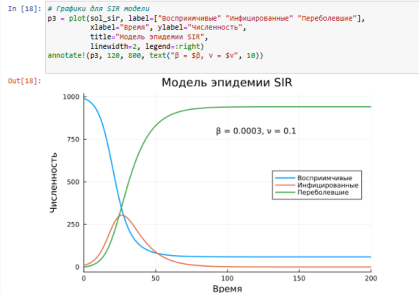


Рис. 20: График модели эпидемии SIR

Задания для самостоятельного выполнения

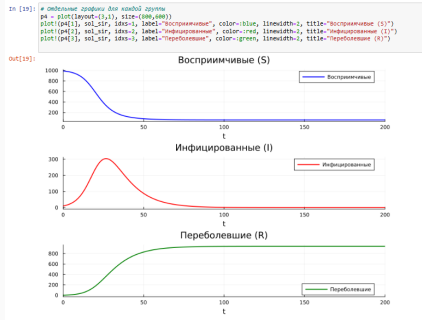


Рис. 21: Отдельные графики модели эпидемии SIR

Задания для самостоятельного выполнения

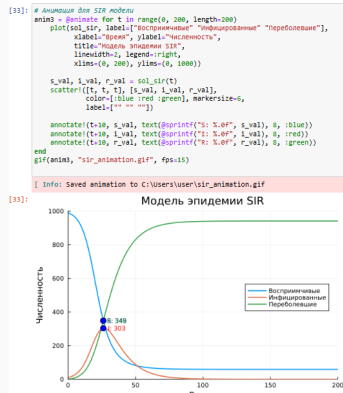


Рис. 22: Анимация модели эпидемии SIR

Задания для самостоятельного выполнения

4. Как расширение модели SIR (Susceptible-Infected-Removed) по результатам эпидемии испанки была предложена модель SEIR (Susceptible-Exposed-Infected-Removed):

$$\begin{aligned}\dot{s}(t) &= -\frac{\beta}{N}s(t)i(t), \\ \dot{e}(t) &= \frac{\beta}{N}s(t)i(t) - \delta e(t), \\ \dot{i}(t) &= \delta e(t) - \gamma i(t), \\ \dot{r}(t) &= \gamma i(t).\end{aligned}$$

Размер популяции сохраняется:

Задания для самостоятельного выполнения

```
In [1]: using LinearAlgebra

In [8]: # 4. Модель SEIR
function seir_model!(du, u, p, t)
    S, E, I, R = u
    β, θ, γ, N = p

    du[1] = -β/N * S * I           # dS/dt
    du[2] = β/N * S * I - δ * E   # dE/dt
    du[3] = δ * E - γ * I         # dI/dt
    du[4] = γ * I                 # dR/dt
end

Out[8]: seir_model! (generic function with 1 method)

In [9]: # параметры для SEIR модели
β_seir = 0.4           # коэффициент заражения
δ = 0.2               # скорость перехода из экспонированных в инфицированные (1/инкубационный период)
γ = 0.1               # скорость выздоровления
N = 1000              # общая численность популяции
u0_seir = [990.0, 5.0, 5.0, 0.0] # [S, E, I, R]
tspan_seir = (0.0, 200.0)
p_seir = [β_seir, δ, γ, N]

# решение SEIR модели
prob_seir = ODEProblem(seir_model!, u0_seir, tspan_seir, p_seir)
sol_seir = solve(prob_seir, Tsitsis())

Out[9]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 39-element Vector{Float64}:
 0.0
 0.0034636383408691634
 0.03810082174956079
 0.3240402380575046
 1.0429182531932037
 2.1449881960425676
 3.546109256096558
 5.408499196047138
 7.756465947281605
10.725693127037474
14.266135808642983
18.244135079256545
22.511394425638896
⋮
107.7468492923399
114.98622735933296
122.6176530598499
130.64462964300764
139.0182631069364
147.6904495203093
156.61540551835887
⋮
```

Рис. 23: Модель SEIR

Задания для самостоятельного выполнения

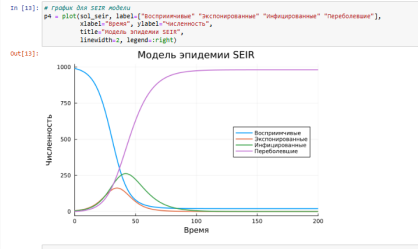


Рис. 24: График модели SEIR

Задания для самостоятельного выполнения

```
In [21]: # Сравнение SIR и SEIR
# Пересчитаем SIR с аналогичными параметрами для сравнения
beta_sir = 0.4
v = 0.1
u0_sir_comp = [990.0, 10.0, 0.0]
p_sir_comp = [0_sir, v]

prob_sir_comp = odeProblem(sir_model1, u0_sir_comp, tspan_seir, p_sir_comp)
sol_sir_comp = solve(prob_sir_comp, Tsits())

Out[21]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 1378-element Vector{Float64}:
 0.0
 0.0011148574399352122
 0.002427920030841821
 0.003945915418994764
 0.005583244431774284
 0.007498676528885386
 0.009663988012135488
 0.012137185432393652
 0.015473371868579496
 0.018871901893894805
 0.022809316927480615
 0.0273835889575277665
 0.0325423289622347638
 0.0383835423289622347638
 0.04482264372444
 0.051861180799299
 0.0595008644492744
 0.06774962793432192
 0.0765081831944342
 0.08577635632215157
 0.09555424674161383
 0.1058428167822162
 0.11664081621828732
 0.1279483982103294
 0.1397664326185997
 0.152094080799299
 0.16493123864233578
 0.1782779283818105
 0.192135993127683
 0.20650863179951418
 0.2213939373258057
 0.2367939373258057
 0.25270864526
 0.269135993127683
 0.2860779283818105
 0.303535993127683
 0.32150863179951418
 0.34000000000000006
 0.35902777777777777
 0.37859375000000006
 0.3986986111111111
 0.41934277777777777
 0.44052700000000006
 0.46225222222222225
 0.4845188888888889
 0.5073170000000001
 0.5306666666666667
 0.5545677777777778
 0.5790214285714286
 0.6040385714285714
 0.6296192857142857
 0.6557735714285714
 0.6825014285714286
 0.7098127777777778
 0.7377076190476191
 0.7661869047619048
 0.7952506190476191
 0.8248986190476191
 0.8551306190476191
 0.8859464285714286
 0.9173469047619048
 0.9493321428571429
 0.9819030952380952
 1.0150595238095238
 1.0487914285714286
 1.0830985714285714
 1.1179809523809524
 1.1534385714285714
 1.1894714285714286
 1.2260795238095238
 1.2632628571428571
 1.3010314285714286
 1.3393728571428571
 1.3782937500000001
 1.4177942857142857
 1.4578742857142857
 1.4985337500000001
 1.5397728571428571
 1.5815914285714286
 1.6239904761904762
 1.6669690476190476
 1.7105271428571429
 1.7546642857142857
 1.7993804761904762
 1.8446757142857143
 1.8905509523809524
 1.9370061904761905
 1.9840414285714286
 2.0316561904761905
 2.0798414285714286
 2.1286061904761905
 2.1779509523809524
 2.2278752380952381
 2.2783785714285714
 2.3294609523809524
 2.3811321428571429
 2.4333928571428571
 2.4862461904761905
 2.5396995238095238
 2.5937528571428571
 2.6484061904761905
 2.7036595238095238
 2.7595128571428571
 2.8159661904761905
 2.8729261904761905
 2.9304969047619048
 2.9886742857142857
 3.0474585714285714
 3.1068495238095238
 3.1668471428571429
 3.2274514285714286
 3.2886614285714286
 3.3504771428571429
 3.4128985714285714
 3.4759252380952381
 3.5395571428571429
 3.6037942857142857
 3.6686361904761905
 3.7340828571428571
 3.8001342857142857
 3.8667909523809524
 3.9340528571428571
 4.0019209523809524
 4.0703942857142857
 4.1394728571428571
 4.2091561904761905
 4.2794428571428571
 4.3503328571428571
 4.4218285714285714
 4.4939342857142857
 4.5666509523809524
 4.6399785714285714
 4.7139171428571429
 4.7884669047619048
 4.8636276190476191
 4.9393992380952381
 5.0157821428571429
 5.0927752380952381
 5.1703785714285714
 5.2485928571428571
 5.3274171428571429
 5.4068514285714286
 5.4868952380952381
 5.5675485714285714
 5.6488109523809524
 5.7306821428571429
 5.8131628571428571
 5.8962542857142857
 5.9799561904761905
 6.0642685714285714
 6.1491914285714286
 6.2347242857142857
 6.3208661904761905
 6.4076171428571429
 6.4949769047619048
 6.5829452380952381
 6.6715221428571429
 6.7607069047619048
 6.8504992380952381
 6.9408995238095238
 7.0319069047619048
 7.1235214285714286
 7.2157428571428571
 7.3085709523809524
 7.4019052380952381
 7.4958452380952381
 7.5903909523809524
 7.6855428571428571
 7.7812995238095238
 7.8776669047619048
 7.9746452380952381
 8.0722352380952381
 8.1704352380952381
 8.2692452380952381
 8.3686652380952381
 8.4686952380952381
 8.5693352380952381
 8.6705852380952381
 8.7724452380952381
 8.8749142857142857
 8.9779928571428571
 9.0816809523809524
 9.1859769047619048
 9.2908809523809524
 9.3963928571428571
 9.5025128571428571
 9.6092509523809524
 9.7166069047619048
 9.8245809523809524
 9.9331714285714286
 10.042378571428571
 10.152200952380952
 10.262638571428571
 10.373691428571429
 10.485359523809524
 10.597642857142857
 10.710540952380952
 10.824054285714286
 10.938182857142857
 11.052925238095238
 11.168280952380952
 11.284249523809524
 11.400830952380952
 11.518024285714286
 11.635830952380952
 11.754249523809524
 11.873280952380952
 11.992925238095238
 12.113184285714286
 12.2340569047619048
 12.355542857142857
 12.477642857142857
 12.599348571428571
 12.721664285714286
 12.844590952380952
 12.968122857142857
 13.092260952380952
 13.217004285714286
 13.342352857142857
 13.4683069047619048
 13.5948669047619048
 13.722039523809524
 13.849820952380952
 13.978210952380952
 14.107211428571429
 14.236822857142857
 14.367045238095238
 14.497878571428571
 14.629322857142857
 14.7613869047619048
 14.894050952380952
 15.027324285714286
 15.1612069047619048
 15.295698571428571
 15.430808571428571
 15.5665369047619048
 15.702883571428571
 15.839848571428571
 15.977432857142857
 16.1156369047619048
 16.254459523809524
 16.393890952380952
 16.533939523809524
 16.6745969047619048
 16.815872857142857
 16.957767142857143
 17.099270952380952
 17.241383571428571
 17.384105238095238
 17.527445952380952
 17.671404285714286
 17.815980952380952
 17.961174285714286
 18.106984285714286
 18.253415238095238
 18.400463571428571
 18.548130952380952
 18.696413571428571
 18.845312857142857
 18.994827142857143
 19.1449569047619048
 19.295699523809524
 19.4470569047619048
 19.598917142857143
 19.751340952380952
 19.9043369047619048
 20.057874285714286
 20.211922857142857
 20.366582857142857
 20.521850952380952
 20.677678571428571
 20.834065238095238
 20.991010952380952
 21.148514285714286
 21.3065769047619048
 21.465198571428571
 21.624379523809524
 21.784219523809524
 21.944618571428571
 22.1055769047619048
 22.267094285714286
 22.429170952380952
 22.5918069047619048
 22.755003571428571
 22.918760952380952
 23.083078571428571
 23.247954285714286
 23.413392857142857
 23.579384285714286
 23.745929523809524
 23.913028571428571
 24.080680952380952
 24.248984285714286
 24.417839523809524
 24.587254285714286
 24.757228571428571
 24.927760952380952
 25.098852857142857
 25.270493571428571
 25.442683571428571
 25.615423571428571
 25.788713571428571
 25.962553571428571
 26.136943571428571
 26.311883571428571
 26.487373571428571
 26.663413571428571
 26.839993571428571
 27.017124285714286
 27.194804285714286
 27.373034285714286
 27.551813571428571
 27.731142857142857
 27.911022857142857
 28.091452857142857
 28.272433571428571
 28.453964285714286
 28.636044285714286
 28.818673571428571
 29.001851428571429
 29.185578571428571
 29.369853571428571
 29.5546769047619048
 29.739948571428571
 29.925768571428571
 30.112527142857143
 30.299834285714286
 30.487689523809524
 30.676093571428571
 30.865045238095238
 31.054544285714286
 31.244590952380952
 31.435184285714286
 31.626324285714286
 31.818019523809524
 32.010269523809524
 32.203073571428571
 32.396430952380952
 32.590340952380952
 32.784804285714286
 32.979822857142857
 33.175395238095238
 33.371522857142857
 33.568204285714286
 33.765440952380952
 33.963232857142857
 34.161570952380952
 34.360460952380952
 34.559892857142857
 34.7598769047619048
 34.960413571428571
 35.161502857142857
 35.363143571428571
 35.5653369047619048
 35.768082857142857
 35.971380952380952
 36.175229523809524
 36.379628571428571
 36.584577142857143
 36.7900769047619048
 36.9961269047619048
 37.202727142857143
 37.409877142857143
 37.6175769047619048
 37.825825238095238
 38.034631428571429
 38.243995238095238
 38.4539169047619048
 38.6643969047619048
 38.8754369047619048
 39.0870369047619048
 39.2991969047619048
 39.5119169047619048
 39.7252969047619048
 39.9398369047619048
 40.1548369047619048
 40.3702969047619048
 40.5863169047619048
 40.8028969047619048
 41.0200369047619048
 41.237732857142857
 41.455984285714286
 41.674790952380952
 41.894152857142857
 42.114069523809524
 42.334540952380952
 42.5555669047619048
 42.777147142857143
 42.999281428571429
 43.221968571428571
 43.445208571428571
 43.669001428571429
 43.8933469047619048
 44.118244285714286
 44.343693571428571
 44.569694285714286
 44.7962469047619048
 45.023350952380952
 45.2509069047619048
 45.478914285714286
 45.707374285714286
 45.936285714285714
 46.165647142857143
 46.395458571428571
 46.625719523809524
 46.856430952380952
 47.087592857142857
 47.3191969047619048
 47.551342857142857
 47.784030952380952
 48.017260952380952
 48.251142857142857
 48.4855769047619048
 48.7205669047619048
 48.956112857142857
 49.192204285714286
 49.428841428571429
 49.666023571428571
 49.903751428571429
 50.142034285714286
 50.380871428571429
 50.620262857142857
 50.860208571428571
 51.100708571428571
 51.341762857142857
 51.583370952380952
 51.825522857142857
 52.068224285714286
 52.311475238095238
 52.5552769047619048
 52.799628571428571
 53.044539523809524
 53.290000952380952
 53.536012857142857
 53.782574285714286
 54.029685714285714
 54.2773469047619048
 54.525557142857143
 54.774317142857143
 55.0236269047619048
 55.273485238095238
 55.523892857142857
 55.774848571428571
 56.026352857142857
 56.2784069047619048
 56.531010952380952
 56.784164285714286
 57.037867142857143
 57.292118571428571
 57.546918571428571
 57.802268571428571
 58.058167142857143
 58.314614285714286
 58.571608571428571
 58.829150952380952
 59.087241428571429
 59.345880952380952
 59.605068571428571
 59.864805238095238
 60.125089523809524
 60.3859169047619048
 60.647290952380952
 60.909172857142857
 61.171580952380952
 61.434539523809524
 61.698048571428571
 61.962108571428571
 62.226719523809524
 62.491880952380952
 62.757592857142857
 63.0238569047619048
 63.290671428571429
 63.5580369047619048
 63.825952857142857
 64.094419523809524
 64.3633469047619048
 64.632834285714286
 64.902871428571429
 65.173458571428571
 65.444594285714286
 65.716279523809524
 65.988514285714286
 66.261308571428571
 66.534651428571429
 66.808542857142857
 67.082980952380952
 67.357974285714286
 67.633522857142857
 67.909624285714286
 68.186275238095238
 68.463475238095238
 68.741224285714286
 69.019522857142857
 69.298379523809524
 69.5777369047619048
 69.857694285714286
 70.138251428571429
 70.419408571428571
 70.701164285714286
 70.983519523809524
 71.266474285714286
 71.549928571428571
 71.833891428571429
 72.118352857142857
 72.403413571428571
 72.689073571428571
 72.975332857142857
 73.262190952380952
 73.549648571428571
 73.837704285714286
 74.126358571428571
 74.415612857142857
 74.7054669047619048
 74.995930952380952
 75.287003571428571
 75.578684285714286
 75.870973571428571
 76.163871428571429
 76.457377142857143
 76.751490952380952
 77.046212857142857
 77.341543571428571
 77.637483571428571
 77.934032857142857
 78.231190952380952
 78.528957142857143
 78.827331428571429
 79.126312857142857
 79.425891428571429
 79.726068571428571
 80.026843571428571
 80.328217142857143
 80.630189523809524
 80.932760952380952
 81.235931428571429
 81.539701428571429
 81.844070952380952
 82.149040952380952
 82.454611428571429
 82.760782857142857
 83.067554285714286
 83.374925238095238
 83.682895238095238
 83.991464285714286
 84.300631428571429
 84.6103969047619048
 84.920760952380952
 85.231722857142857
 85.543282857142857
 85.855441428571429
 86.168198571428571
 86.481553571428571
 86.7955069047619048
 87.110058571428571
 87.425208571428571
 87.7409569047619048
 88.057303571428571
 88.374248571428571
 88.691791428571429
 89.009932857142857
 89.328671428571429
 89.647904285714286
 89.967731428571429
 90.288152857142857
 90.609168571428571
 90.930778571428571
 91.252981428571429
 91.5757769047619048
 91.899164285714286
 92.223143571428571
 92.547612857142857
 92.872182857142857
 93.197853571428571
 93.523624285714286
 93.849494285714286
 94.175464285714286
 94.501533571428571
 94.827701428571429
 95.153967142857143
 95.480258571428571
 95.8066069047619048
 96.133011428571429
 96.459471428571429
 96.7859869047619048
 97.112557142857143
 97.439181428571429
 97.765858571428571
 98.0925869047619048
 98.419371428571429
 98.746211428571429
 99.0730869047619048
 99.400012857142857
 99.726989523809524
 100.054017142857143
 100.381094285714286
 100.708224285714286
 101.0354069047619048
 101.362641428571429
 101.689927142857143
 102.017263571428571
 102.344650952380952
 102.672088571428571
 102.9995769047619048
 
```

Задания для самостоятельного выполнения

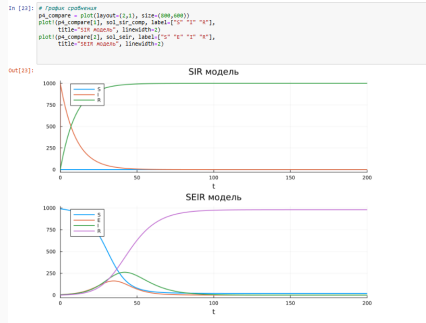


Рис. 26: Графики сравнения SIR и SEIR

Задания для самостоятельного выполнения

```
In [24]: # Anonus SEIR vs SIR
peak_i_seir, t_peak_seir = findmax(sol_seir[3,:])
peak_i_sir, t_peak_sir = findmax(sol_sir_comp[2,:])

Out[24]: (997.6785241468378, 16)

In [25]: println("SIR модель:")
println("Пик инфекции: ${round(peak_i_sir, digits=1)} в момент t=${round(t_peak_sir, digits=1)}")
println("SEIR модель:")
println("Пик инфекции: ${round(peak_i_seir, digits=1)} в момент t=${round(t_peak_seir, digits=1)}")
println("Задержка пика в SEIR: ${round(t_peak_seir - t_peak_sir, digits=1)} единиц времени")

SIR модель:
Пик инфекции: 997.7 в момент t=16.0
SEIR модель:
Пик инфекции: 262.1 в момент t=17.0
Задержка пика в SEIR: 1.0 единиц времени
```

Рис. 27: Анализ результатов

5. Для дискретной модели Лотки–Вольтерры:

$$\begin{aligned}X_1(t+1) &= aX_1(t)(1 - X_1(t)) - X_1(t)X_2(t), \\X_2(t+1) &= -cX_2(t) + dX_1(t)X_2(t).\end{aligned}$$

с начальными данными $a = 2, c = 1, d = 5$ найдите точку равновесия. Получите и сравните аналитическое и численное решения. Численное решение изобразите на фазовом портрете.

Задания для самостоятельного выполнения

```
In [26]: # С. дискретная модель Лотки-Вольтерры
function lotka_voltterra_discrete(x, a, c, d)
    x1, x2 = x
    x1_next = a * x1 * (1 - x1) - x1 * x2
    x2_next = -c * x2 + d * x1 * x2
    return [x1_next, x2_next]
end

Out[26]: lotka_voltterra_discrete (generic function with 1 method)

In [27]: # Параметры
a = 2.0
c = 1.0
d = 3.0

Out[27]: 5.0

In [28]: # Аналитическое нахождение точек равновесия
# решим систему
eq1 = [0.0, 0.0]

# Точка равновесия 2: нахождение из системы уравнений
x1_eq = (1 + c) / d
x2_eq = a * (1 - x1_eq) * x1_eq
eq2 = [x1_eq, x2_eq]

Out[28]: 2-element Vector{Float64}:
 0.4
 0.19999999999999996

In [29]: # Численное решение
function simulate_lotka_voltterra(t, x0, a, c, d)
    trajectory = zeros(2, T)
    trajectory[:, 1] = x0

    for t in 1:T-1
        trajectory[:, t+1] = lotka_voltterra_discrete(trajectory[:, t], a, c, d)
    end

    return trajectory
end

Out[29]: simulate_lotka_voltterra (generic function with 1 method)

In [30]: # Симуляция
T = 1000
x0_1 = [0.1, 0.1] # начальные условия близко к неустойчивой точке равновесия
x0_2 = [0.5, 0.2] # другие начальные условия

traj1 = simulate_lotka_voltterra(T, x0_1, a, c, d)
traj2 = simulate_lotka_voltterra(T, x0_2, a, c, d)

Out[30]: 2x1000 Matrix{Float64}:
 0.5  0.4  0.36  0.3528  0.371992  0.4  0.4  0.4  0.4  0.4  0.4  0.4
 0.2  0.2  0.2  0.24  0.18226  0.2  0.2  0.2  0.2  0.2  0.2  0.2
```

Рис. 28: Модель Лотки-Вольтерры

Задания для самостоятельного выполнения

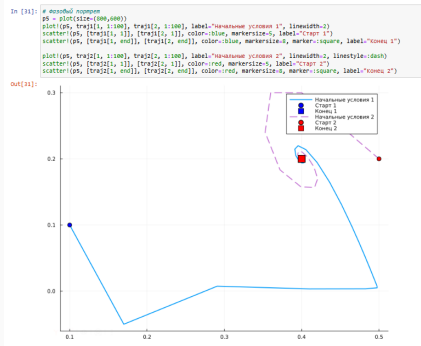


Рис. 29: Фазовый портрет модели Лотки-Вольтерры

Задания для самостоятельного выполнения

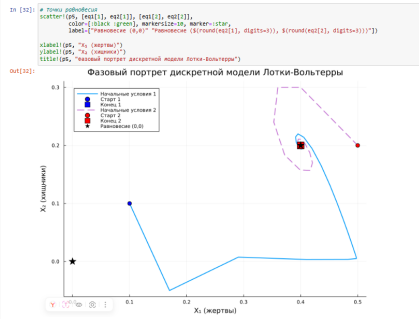


Рис. 30: Фазовый портрет с точками равновесия

Задания для самостоятельного выполнения

```
In [10]: # Проверка сходимости к точке равновесия
final_states = traj[i], end
final_states = traj[i], end

print("\nПроверка сходимости к точке равновесия:")
print("Траектория 1: конечное состояние = (%.4f, %.4f)" % (round(final_states[1], 4), round(final_states[2], 4)))
print("Траектория 2: конечное состояние = (%.4f, %.4f)" % (round(final_states[1], 4), round(final_states[2], 4)))
print("Точка равновесия: (%.4f, %.4f)" % (round(eq[1], 4), round(eq[2], 4)))

Проверка сходимости к точке равновесия:
Траектория 1: конечное состояние = (0.4, 0.2)
Траектория 2: конечное состояние = (0.4, 0.2)
Точка равновесия: (0.4, 0.2)
```

Рис. 31: Проверка сходимости к точке равновесия

6. Реализовать на языке Julia модель отбора на основе конкурентных отношений:

$$\dot{x} = \alpha x - \beta xy,$$

$$\dot{y} = \alpha y - \beta xy.$$

Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

Задания для самостоятельного выполнения

```
In [56]: # 6. Модель конкурентных отношений
# Параметры модели
a = 1.0 # скорость роста
b = 0.5 # интенсивность конкуренции
x_0 = 2.0 # начальная численность вида x
y_0 = 1.0 # начальная численность вида y
t_0 = 0.0
t_end = 10.0

Out[56]: 10.0

In [56]: # определение системы ODE
function competition!(du, u, p, t)
    x, y = u
    a, b = p
    du[1] = a * x - b * x * y # dx/dt
    du[2] = -a * y - b * x * y # dy/dt
end

# начальные условия и параметры
u0 = [x_0, y_0]
p = [a, b]
tspan = (t_0, t_end)

# решение задачи
prob = ODEProblem(competition!, u0, tspan, p)
sol = solve(prob, Tsit5(), dt=0.01)

Out[56]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 3153-element vector{Float64}:
 0.0
 0.01
 0.10999999999999999
 0.37678003409997727
 0.7232762200760046
 1.1135034307656617
 1.5403370211743245
 1.9992782626408304
 2.526642080978965
 2.56785117922192
 2.7381644115159587
 2.807630956077937
 3.027713083406258
 ...
 9.996000050019963
 9.997119053821104
 9.997439155614666
 9.997758355482373
 9.998077453510023
 9.998396444973521
 9.998715344244195
 9.999034137873540
 9.999352828309267
 9.999671417980683
 9.9999898659747
 10.0
u: 3153-element vector{Vector{Float64}}:
 [2.0, 1.0]
 [2.0100001255742214, 0.9999749504000531]
 [2.11319997730996, 0.9999219068276245]
```

Рис. 32: Модель конкурентных отношений

Задания для самостоятельного выполнения

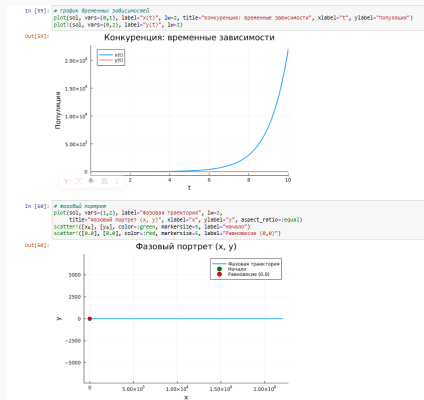


Рис. 33: График временных зависимостей и фазовый портрет

Задания для самостоятельного выполнения

```
in [43]: # Анимация геофизического процесса
anim = @animate for i in 1:length(sol.t)
    plot(sol, vars=(1:2), label="температура", lw=2,
         title="Анализ: изменение по газовой мощности", xlabel="x", ylabel="y", aspect_ratio=:equal)
    scatter([sol.u[i][1]], [sol.u[i][2]], color=:red, markersize=5, label="темная вода")
    scatter([x4], [y4], color=:green, markersize=5, label="темная")
end

out[43]: animation("C:\\Users\\user\\AppData\\Local\\Temp\\15128FF9", ["000001.png", "000002.png", "000003.png", "000004.png", "000005.png", "000006.png", "000007.png", "000008.png", "000009.png", "000010.png", "000011.png", "000012.png", "000013.png", "000014.png", "000015.png", "000016.png", "000017.png", "000018.png", "000019.png", "000020.png", "000021.png", "000022.png", "000023.png", "000024.png", "000025.png", "000026.png", "000027.png", "000028.png", "000029.png", "000030.png", "000031.png", "000032.png", "000033.png", "000034.png", "000035.png", "000036.png", "000037.png", "000038.png", "000039.png", "000040.png", "000041.png", "000042.png", "000043.png", "000044.png", "000045.png", "000046.png", "000047.png", "000048.png", "000049.png", "000050.png", "000051.png", "000052.png", "000053.png", "000054.png", "000055.png", "000056.png", "000057.png", "000058.png", "000059.png", "000060.png", "000061.png", "000062.png", "000063.png", "000064.png", "000065.png", "000066.png", "000067.png", "000068.png", "000069.png", "000070.png", "000071.png", "000072.png", "000073.png", "000074.png", "000075.png", "000076.png", "000077.png", "000078.png", "000079.png", "000080.png", "000081.png", "000082.png", "000083.png", "000084.png", "000085.png", "000086.png", "000087.png", "000088.png", "000089.png", "000090.png", "000091.png", "000092.png", "000093.png", "000094.png", "000095.png", "000096.png", "000097.png", "000098.png", "000099.png", "000100.png"])
anim
```

Рис. 34: Анимация модели конкурентных отношений

7. Реализовать на языке Julia модель консервативного гармонического осциллятора:

$$\ddot{x} + \omega_0^2 x = 0, \quad x(t_0) = x_0, \quad \dot{x}(t_0) = y_0,$$

где ω_0 — циклическая частота. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

Задания для самостоятельного выполнения

```
In [47]: # # Параметры
w_0 = 2H + 1.0 # циклическая частота (например, 1 Гц)
u_0 = 1.0 # начальное положение
t_0 = 0.0 # начальная скорость
t_end = 5.0 # начальное время
# конечное время

# Определение системы ODE
function harmonic_oscillator!(du, u, p, t)
    du[1] = u[2]
    du[2] = -p[1]^2 * u[1]
end

# Начальные условия
u0 = [u_0, 0]
p = [w_0]
tspan = (t_0, t_end)

Out[47]: (0.0, 5.0)

In [48]: prob = ODEProblem(harmonic_oscillator!, u0, tspan, p)
sol = solve(prob, Tsit5(), dt=0.01)

Out[48]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 48-element Vector{Float64}:
 0.0
 0.01
 0.04080817638990946
 0.07100815210080656
 0.11796565175557918
 0.1784313164466131
 0.24937951261455662
 0.32884951255679724
 0.409453231238024084
 0.5076610856158231
 0.6067957126541759
 0.7107892498112556
 0.828976057472839
 3.5551799663117357
 3.6835282345853866
 3.8231398731515101
 3.9682262911823977
 4.09235648515142785
 4.2081620182389925
 4.2623956809508445
 4.3022432680515384
 4.3353568098554436
 4.7606009218715533
 4.984069356885321
 5.0
u: 48-element Vector{Vector{Float64}}:
 [0.0, 0.0]
 [0.998787288455999, 0.39452446974157385]
 [1.7776237888185577, -1.3322280255508793]
```

Рис. 35: Консервативный гармонический осциллятор

Задания для самостоятельного выполнения

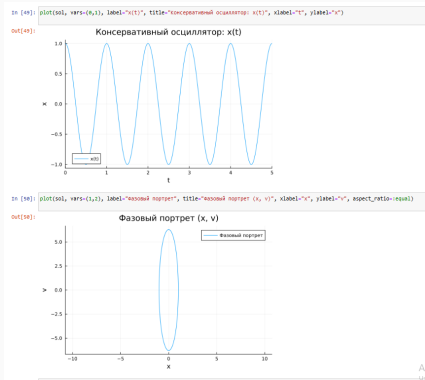


Рис. 36: График консервативного гармонического осциллятора и фазовый портрет

Задания для самостоятельного выполнения



Рис. 37: Анимация фазового портрета

8. Реализовать на языке Julia модель свободных колебаний гармонического осциллятора:

$$\ddot{x} + 2\gamma\dot{x} + \omega_0^2 x = 0, \quad x(t_0) = x_0, \quad \dot{x}(t_0) = y_0,$$

где ω_0 — циклическая частота, γ — параметр, характеризующий потери энергии. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

Задания для самостоятельного выполнения

```
In [51]: # Параметры
u_0 = 2*pi * 1.0 # циклическая частота
gamma = 0.3 # коэффициент затухания (подбираем для слабого затухания)
x_0 = 1.0 # начальное положение
y_0 = 0.0 # начальная скорость
t_0 = 0.0
t_end = 10.0

# определение системы ОДУ
function damped_oscillator(du, u, p, t)
    du[1] = u[2]
    du[2] = -2 * pi^2 * u[1] - gamma * u[2]
end

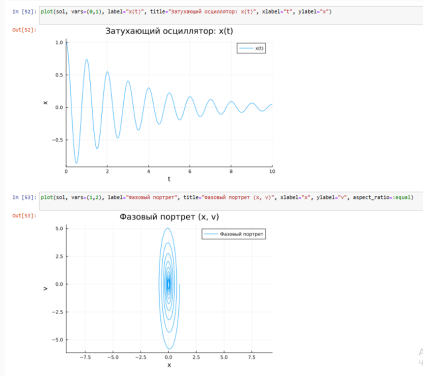
# начальные условия
u0 = [x_0, y_0]
p = [gamma, u_0]
tspan = (t_0, t_end)

# создание задачи и ее решение
prob = ODEProblem{damped_oscillator, u0, tspan, p}
sol = solve(prob, Tsit5(), dt=0.01)

Out[51]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
ti: 83-element Vector{Float64}:
 0.0
 0.01
 0.034063953795469218
 0.07119871188563381
 0.11043317655556795
 0.17929480405410362
 0.2502283946401078
 0.3260651559174538
 0.4089860777862761
 0.5060697177322483
 0.609521167879406
 0.70945429008112819
 0.8247779513440989
 ...
 0.552492053399936
 0.682670006529662
 0.820294927750208
 0.947853854503938
 0.986879584427475
 9.210192815308595
 9.351832472349045
 9.475339241912419
 9.61080962344573
 9.743622240615747
 9.882577899161241
 10.0
u: 83-element Vector{Vector{Float64}}:
 [1.0, 0.0]
 [0.998930668795619, -0.3933432600830469]
 [0.977377012106171 -1.171001752472209e1]
```

Рис. 38: Свободные колебания гармонического осциллятора

Задания для самостоятельного выполнения



Ал
Чис

Рис. 39: График затухающего гармонического осциллятора и его фазовый портрет

Задания для самостоятельного выполнения

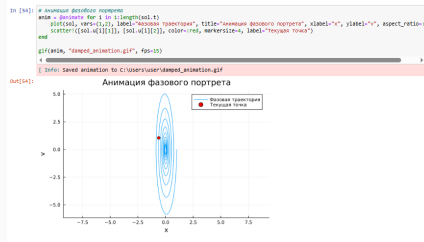


Рис. 40: Анимация фазового портрета

Выводы

В результате выполнения данной лабораторной работы я освоила специализированные пакеты для решения задач в непрерывном и дискретном времени.